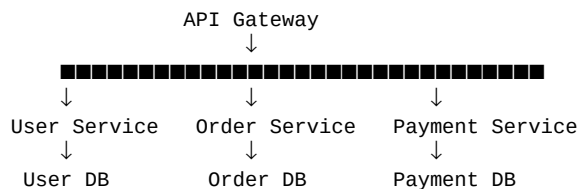


Microservices Architecture — Practical System Design Notes

Hinglish • Service Boundaries • Communication • Database-per-Service • Saga • Outbox • Resilience

1. Microservices Kya Hai?

Microservices architecture mein application ko independently deployable services mein divide kiya jata hai. Har service ek clear business capability own karti hai aur ideally independently scale/deploy ho sakti hai.



2. Important: Microservices $\neq$ Multiple Servers

Sirf 5 Node.js servers bana dene se microservices architecture nahi ban jaata. Important hai service ownership, boundaries, independent deployment, data ownership aur controlled communication.

Bad:
5 services
↓
one shared DB
↓
all tables accessible by everyone

Better:
User → User DB
Order → Order DB
Payment → Payment DB

3. Monolith vs Microservices

Monolith	Microservices
One deployable application	Multiple independently deployable services
Usually simpler initially	More operational complexity
Easy local development	Distributed debugging
Shared process/database often	Service ownership
Scaling often whole app	Scale individual service

4. When Microservices Make Sense

- Different business domains need independent scaling.
- Teams need independent deployments.
- Some workloads have very different resource profiles.
- Failure isolation is important.
- Organization has enough engineering/operations maturity.

Small application ko unnecessarily microservices mein todna operational overhead create kar sakta hai.

5. Service Boundary

Service boundary ideally business capability ke around define karo, technical layer ke around nahi.

Good:

User Service
Order Service
Payment Service
Inventory Service

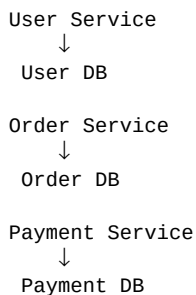
Less useful:

Controller Service
Database Service
Validation Service

6. Domain Ownership

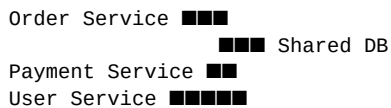
Order Service ko orders ka business logic aur lifecycle own karna chahiye. Payment Service payment state own kare. Dusri services APIs/events ke through interact karein.

7. Database per Service



Database-per-service strong ownership boundary provide karta hai. Ek service doosri service ke tables directly update nahi karti.

8. Shared Database Problem



Problem:

schema coupling
deployment coupling
ownership unclear
cross-service queries

Shared DB kabhi transitional architecture mein exist kar sakta hai, but long-term independent services ke benefits reduce kar deta hai.

9. Service-to-Service Communication

Do broad patterns: synchronous request/response aur asynchronous messaging.

Synchronous:

Order → Payment API → response

Asynchronous:

Order → Queue/Event → Payment Worker

10. REST

REST/HTTP simple, familiar aur easy to debug hai. Public/internal APIs ke liye common choice hai.

POST /payments

GET /orders/123

GET /users/123

11. gRPC

gRPC strongly typed service-to-service communication ke liye useful ho sakta hai, especially internal high-throughput environments mein. Protobuf contracts and streaming capabilities important advantages hain.

12. REST vs gRPC

REST	gRPC
HTTP/JSON common	Protocol Buffers
Easy browser/debug tooling	Strong typed contracts
Simple public APIs	Often strong fit for internal RPC
Human-readable payload	Compact binary payload

13. Synchronous Communication

```
Client
↓
Order Service
↓
Inventory Service
↓
Payment Service
↓
response
```

Simple mental model hai, but dependency chain badhne par latency aur failure propagation increase ho sakti hai.

14. Asynchronous Communication

```
Order Service
↓
Queue
↓
Inventory Worker
↓
Event
↓
Payment Worker
```

Async processing request latency aur coupling reduce kar sakta hai, but consistency, retries, observability aur debugging more complex ho jate hain.

15. Service Discovery

Distributed environment mein services dynamically change ho sakti hain. Service discovery ka purpose service name ko healthy/current instances se resolve karna hai.

```
Order Service
  ↓
payment-service
  ↓
10.0.2.15:8080
10.0.2.19:8080
```

16. API Gateway + Microservices

```
Client
  ↓
WAF / LB
  ↓
API Gateway
  ■→ User
  ■→ Order
  ■→ Payment
  ■→ Search
```

Gateway client-facing concerns handle karta hai; internal service boundaries business capabilities ke around defined hoti hain.

17. Event-Driven Architecture

```
Order Service
  ↓
OrderCreated
  ↓
Message Broker
  ■→ Inventory
  ■→ Notification
  ■→ Analytics
```

One event ko multiple independent consumers process kar sakte hain.

18. Message Broker

Kafka, RabbitMQ, AWS SQS/SNS aur other messaging systems different guarantees/use cases provide karte hain. Selection throughput, ordering, replay, delivery semantics aur operational needs par depend karta hai.

19. Event vs Command

Command:
"CreatePayment"
→ instruction

Event:
"PaymentCreated"
→ fact that something happened

Event consumers ko producer ke internal implementation se loosely coupled rakh sakta hai.

20. Distributed Transaction Problem

Create Order
↓
Reserve Inventory
↓
Charge Payment
↓
Send Notification

Failure:
Payment fails
but inventory already reserved

Now what?

Traditional single-DB transaction across independent services generally practical nahi hoti. Distributed transaction patterns ki zaroorat padti hai.

21. Saga Pattern

Saga long-running business transaction ko multiple local transactions mein break karta hai. Har step ka compensating action ho sakta hai.

Create Order
↓
Reserve Inventory
↓
Charge Payment
↓
Success

If payment fails:
↓
Release Inventory
↓
Cancel Order

22. Saga Choreography

OrderCreated
↓
Inventory Service
↓
InventoryReserved
↓
Payment Service
↓
PaymentFailed
↓
Inventory Service → release

Services events react karke next action trigger karti hain. Central coordinator nahi hota.

23. Saga Orchestration

Saga Orchestrator

- Create Order
- Reserve Inventory
- Charge Payment
- Compensate on failure

Central orchestrator workflow control karta hai. Complex workflows mein visibility better ho sakti hai, but orchestrator itself important component ban jata hai.

24. Choreography vs Orchestration

Choreography	Orchestration
Event-driven	Central coordinator
Less central control	Clear workflow visibility
Can become hard to follow at scale	Orchestrator can become complex
Good for simpler event flows	Useful for complex workflows

25. Outbox Pattern

Classic problem: DB transaction commit ho gaya but event publish fail ho gaya. Outbox pattern event ko same DB transaction mein outbox table mein save karta hai.

```
BEGIN TRANSACTION
```

```
UPDATE orders  
INSERT INTO outbox(event)
```

```
COMMIT
```

↓

Outbox Publisher

↓

Queue/Broker

↓

Consumers

Ab DB state aur event intent ek atomic local transaction mein persist hote hain.

26. Idempotency

At-least-once delivery mein same event multiple times process ho sakta hai. Consumer ko idempotent banana important hai.

```
eventId = 123
```

First:

```
process → success  
mark 123 processed
```

Duplicate:

```
eventId 123  
→ already processed  
→ skip
```

27. Delivery Semantics

At-most-once	Message may be lost, no duplicate processing guarantee
At-least-once	Message can be retried/duplicated; consumer must handle duplicates
Exactly-once	End-to-end exactly-once side effects are difficult; do not casually assume it

28. Distributed Consistency

Microservices mein har service ka local transaction atomic ho sakta hai, but overall workflow eventually consistent ho sakta hai. User-facing state machine clearly define karo.

Order:

PENDING

↓

CONFIRMED

↓

COMPLETED

or

PENDING

↓

FAILED

↓

CANCELLED

29. Retry in Microservices

Retry network failures aur temporary dependency failures handle kar sakta hai. But retry storm create na ho, isliye timeout + exponential backoff + jitter + retry budget useful hain.

30. Retry Storm

Service B slow
↓
A retries
↓
more requests to B
↓
B gets even more overloaded
↓
failure increases

Retries failure ko amplify bhi kar sakte hain. Har layer par aggressive retries avoid karo.

31. Circuit Breaker

Service B
↓
repeated failures
↓
Circuit OPEN
↓
fail fast
↓
cooldown
↓
HALF OPEN
↓
test request

Circuit breaker downstream failure ko upstream chain mein spread hone se limit karta hai.

32. Timeout Budget

Agar Gateway timeout 2 seconds hai, to downstream chain ko us budget ke andar fit karna hoga. Nested timeouts poorly configured honge to parent request unnecessarily wait karegi.

33. Bulkhead Pattern

Different workloads/resources ko isolate karne ka idea. Ek failing dependency ke liye unlimited connections use hone se doosre traffic paths starve nahi hone chahiye.

API

```
■■■■ Pool A → User
■■■■ Pool B → Payment
■■■■ Pool C → Search
```

34. Distributed Tracing

```
Request ID / Trace ID
  ↓
Gateway
  ↓
Order
  ↓
Payment
  ↓
DB
```

Tracing se end-to-end latency breakdown aur failure location identify karna easier hota hai.

35. Observability

- Per-service RPS.
- p50/p95/p99 latency.
- 4xx/5xx errors.
- Dependency timeout rate.
- Circuit breaker opens.
- Queue depth/consumer lag.
- Event processing latency.
- DB latency and connection pool usage.
- Distributed trace duration.

36. Deployment

Microservices independently deploy ki ja sakti hain. Canary, blue/green ya rolling deployment techniques release risk manage karne mein help kar sakti hain.

```
v1 → 90%
v2 → 10%
```

```
Observe:
errors
latency
business metrics
```

Then gradually increase v2

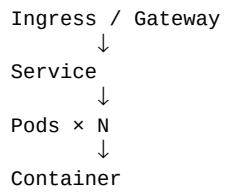
37. Independent Scaling

Traffic:

```
User Service → 2 instances
Order Service → 10 instances
Search Service → 20 instances
Payment      → 5 instances
```

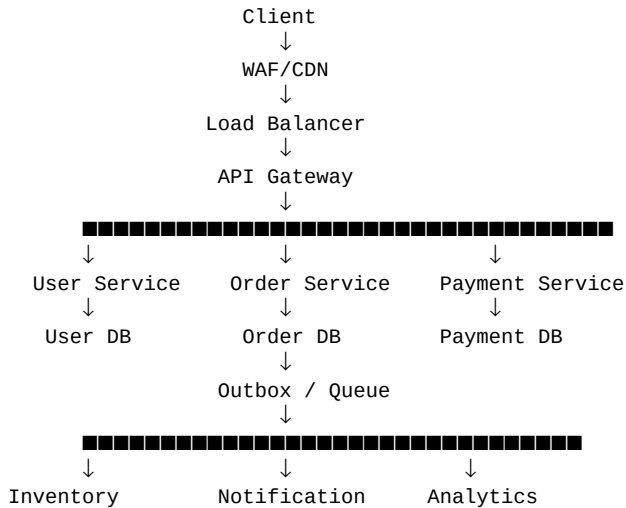
Ye microservices ka major operational benefit hai: workload ke according individual service scale karna.

38. Kubernetes Mental Model



Kubernetes scheduling, service discovery, health checks, rolling deployment aur horizontal scaling jaise platform capabilities provide karta hai. Kubernetes itself microservices ka definition nahi hai.

39. Production Architecture



Shared infrastructure:
Redis / Broker / Observability

40. Failure Scenarios

Failure	Handling
Payment service down	Timeout + circuit breaker + pending state/retry workflow
Duplicate event	Idempotency/event ID deduplication
DB updated, event publish failed	Outbox pattern
Consumer crashes	Retry/redelivery + durable message
One service overloaded	Rate limit + backpressure + independent scaling
Network partition	Timeouts + retries carefully + eventual consistency
Partial Saga failure	Compensating transaction/state transition

41. Local Node.js Microservices Project

Aapke Node.js background ke liye practical project:

API Gateway : 3000

User Service : 3001

Order Service : 3002

Payment Service : 3003

Inventory : 3004

Redis : 6379

RabbitMQ/Kafka : broker

PostgreSQL/MySQL : service DBs

Prometheus : metrics

Grafana : dashboards

- Each service ka separate Express application.
- Each service owns its database/tables.
- Gateway routing + JWT.
- Order creation synchronous.
- OrderCreated event via Outbox.
- Inventory reservation async.

- Payment workflow with idempotency.
- Saga compensation on failure.
- Redis for cache/rate limits.
- Prometheus + Grafana for metrics.
- Docker Compose se locally run karo.

42. Interview Question: Why Not One DB?

Short answer: independent ownership and deployment. Shared DB se schema coupling aur cross-service coordination increase hoti hai. Agar two services same tables directly mutate karein, service boundary weak ho jati hai.

43. Interview Question: How Do You Handle Order + Payment?

Order service order ko PENDING state mein create karega. Payment service idempotent payment attempt handle karegi. Success/failure event ke basis par order state transition hoga. Agar workflow multiple services involve karta hai, Saga + compensation use kar sakte hain.

44. Interview Question: What If Event Is Published Twice?

Consumer event ID/idempotency key store karke duplicate processing skip karega. Side-effecting operations ko unique constraints/state checks ke saath protect karunga.

45. Interview Question: What If Service Is Slow?

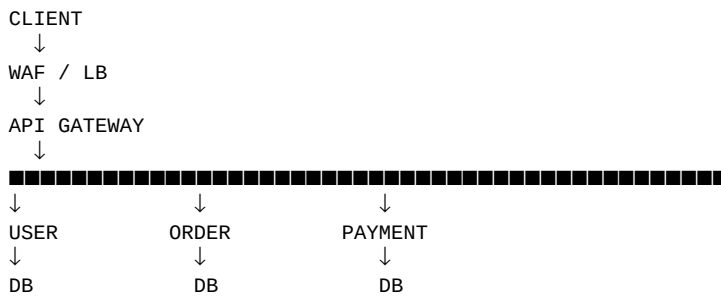
Timeout set karunga, latency trace karunga, retry only where safe, exponential backoff+jitter use karunga, repeated failures par circuit breaker and load shedding apply karunga.

46. Interview Q&A;

Question	Short answer
Microservices kya?	Independently deployable services around clear business capabilities.
Service boundary?	Business/domain capability ke around ownership boundary.
Database per service?	Each service owns its data; others use APIs/events.
REST vs gRPC?	REST simple/common; gRPC strong typed internal RPC use cases ke liye useful.
Sync vs async?	Sync immediate response; async decouples work through messaging.
Saga?	Distributed workflow as local transactions + compensation.
Outbox?	DB transaction mein event intent persist, then publish asynchronously.
Idempotency?	Same operation repeated hone par duplicate side effect avoid karna.
Circuit breaker?	Failing dependency ko repeated calls se protect karna.
Service discovery?	Service name ko healthy/current instances se resolve karna.
Why tracing?	Distributed request ki latency/failure path identify karna.

47. Final Cheat Sheet

MICROSERVICES



Async:
ORDER
↓
OUTBOX
↓
QUEUE/BROKER
↓
INVENTORY / NOTIFICATION / ANALYTICS

Core:
→ clear service boundaries
→ database ownership
→ REST / gRPC
→ sync vs async
→ service discovery
→ events
→ Saga
→ Outbox
→ idempotency
→ retries + timeout
→ circuit breaker
→ tracing
→ independent scaling
→ deployment isolation

Golden interview line:

"I'll define services around business capabilities with clear data ownership. Each service will own its database and expose APIs/events instead of directly modifying another service's data. For distributed workflows I'll use local transactions with Saga compensation, and use the Outbox pattern to reliably publish events. Timeouts, bounded retries, idempotency, circuit breakers and distributed tracing will handle failure and observability."

48. 80/20 Mental Model

Agar interview mein sirf 8 concepts yaad rakhne hain:

1. Service boundary = business capability.
2. Database ownership = service ownership.
3. Gateway = entry point, business logic nahi.
4. Sync calls = simple but tightly coupled.
5. Async events = decoupling + eventual consistency.
6. Saga = distributed workflow + compensation.
7. Outbox = DB update + reliable event publishing.
8. Idempotency + timeout + retry + circuit breaker = failure handling foundation.